

Caching and Data Access Optimization

1. Data Candidates for Caching

To support high read concurrency and maintain low API response times (< 20ms) during traffic surges, cached data is categorized by mutation frequency and computational cost:

Data Type	Mutability	Cache Key Pattern	TTL	Strategy
Movie & Theater Catalog	Low (Static)	catalog:movie:{movie_id}	1 hour	Cache-Aside
Show Details & Timings	Low (Infrequent updates)	show:{show_id}	300 seconds (5 min)	Cache-Aside
Seat Availability Map	High (Rapidly changing)	seats:{show_id}	10 seconds	Short TTL
Dynamic Ticket Pricing	Medium	pricing:{show_id}	120 seconds (2 min)	Cache-Aside

2. Caching Strategy & Access Pattern

We use the **Cache-Aside (Lazy Loading)** pattern for all catalog, show, and seat availability read paths. The application service queries Redis first. If a cache miss occurs, the service queries PostgreSQL, writes the result into Redis with a defined TTL, and returns the response.

Cache-Aside Logic

```
Algorithm: GetShowDetails(show_id)
1. cached = redis.get("show:" + show_id)
2. if cached is not null:
    return cached
3. data = db.query("SELECT * FROM shows WHERE id = ?", show_id)
4. redis.setex("show:" + show_id, 300, data)
5. return data

Algorithm: GetSeatAvailability(show_id)
1. cached = redis.get("seats:" + show_id)
2. if cached is not null:
    return cached
3. data = db.query("SELECT seat_id, status FROM seats WHERE show_id = ?", show_id)
4. redis.setex("seats:" + show_id, 10, data)
5. return data
```

3. Cache Topology & Eviction

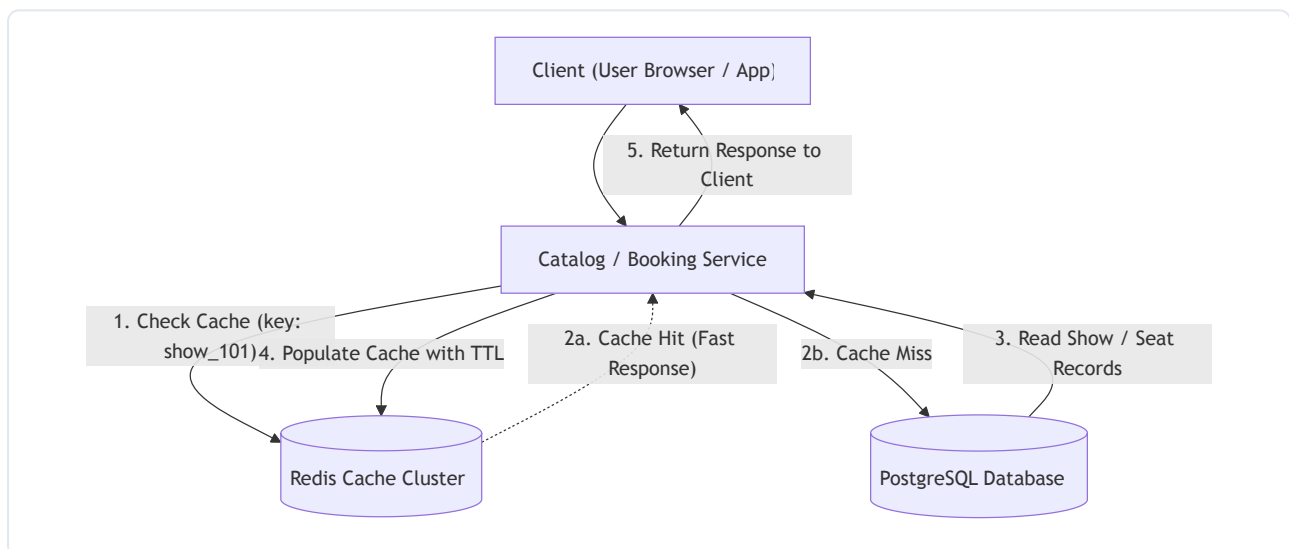
- **Topology:** Distributed Redis Cluster partitioned logically by `city_id`. Since moviegoers search within specific metropolitan areas, caching by city localizes cache lookups and bounds memory usage per shard.
- **Eviction Policy:** Configured with `allkeys-lru` (Least Recently Used). If memory limits are reached, Redis automatically purges cold movie or show records to make room for active, trending releases.

4. TTL & Invalidation Policies

- **Time-To-Live (TTL):** Every cached entry has an explicit TTL to prevent stale data buildup. Show schedules expire after 300 seconds, and seat availability expires after 10 seconds.
- **Active Invalidation on Update:** When theater managers modify show timings or prices via the admin dashboard, the update service executes an explicit cache purge (`redis.delete("show:" + show_id)`), ensuring subsequent user requests immediately fetch fresh data.

5. Cache Interaction Data Flow Diagram

The following data flow diagram illustrates how services interact with the Redis cache cluster and the persistent PostgreSQL database:



6. Cache Consistency & Stale Seat Data Mitigation

During flash sales, hundreds of users view the same screen simultaneously. Completely synchronizing seat status in real time across distributed caches causes severe write amplification. We solve this with a two-tiered consistency model:

1. **Short-Lived Seat Cache (10-second TTL):** The seat layout cache is refreshed every 10 seconds. This shields PostgreSQL from absorbing thousands of repeated layout queries while keeping UI displays accurate within a 10-second window.
2. **Lock as Source of Truth:** The cache is treated as a fast visual guide, not a transactional authority. When a user selects a seat to book, the system bypasses the read cache and attempts the atomic Redis `SETNX` lock directly. If another user has locked the seat, the lock fails immediately, returning a seat unavailable error. Stale cache reads never result in a double booking.